OpenCV 手势识别

作者: 李后军　　时间: 2026-11-12

1. 框架

语言: python

模型: MediaPipe 的手势识别模型

① hand_detector.rknn　　手掌检测模型

② hand_landmarks_detector.rknn　　21 个特征点检测模型

③ gesture_embedder.rknn　　特征嵌入模型

④ canned_gesture_classifier.rknn　　手势分类模型

⑤ palm_detection.rknn　　手掌检测模型

⑥ hand_landmark.rknn　　21 个关键点检测模型

加速处理: RK3588 NPU Core_0 推理加速

说明:

如果使用 NPU 加速,则不能直接使用 MediaPipe 的 API 函数,并没有支持 RK3588 NPU 的 API 函数,只能手写算法.如果使用 CPU,或者使用 GPU 加速,则可以直接使用 MediaPipe 的 API 函数,我这里采用的是 RK3588 的 NPU 加速,所以识别算法只能自己实现,即不用 import mediapipe 即可实现,但是用到的模型现在还是 MediaPipe 的模型.这里采用 rknn-toolkit2+py 脚本,实现 tflite 模型转换为 rknn 模型,下面逐项进行.

2. 模型转换

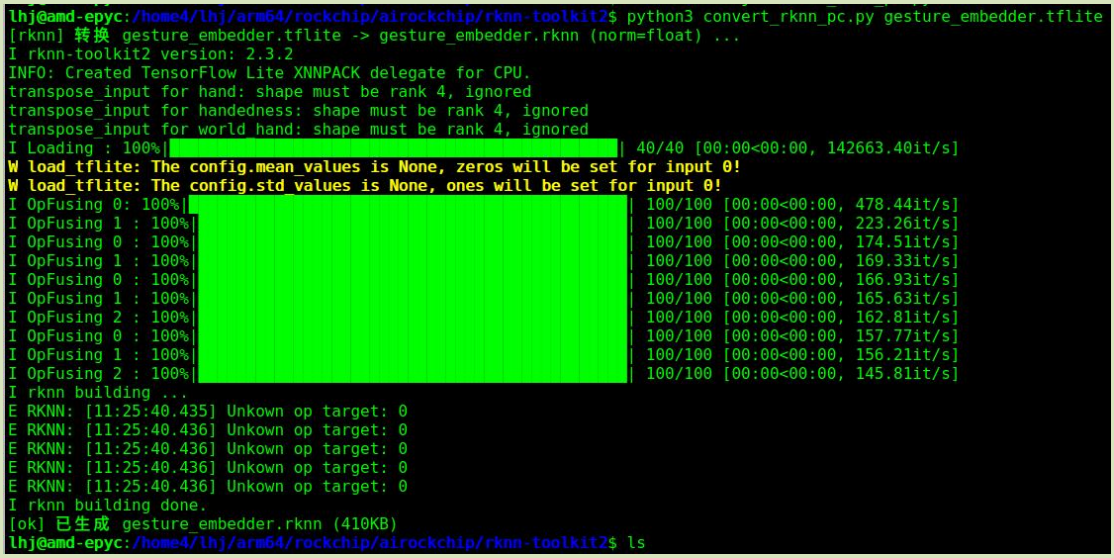
因为需要 rknn-toolkit2,所以首先是拉取源码,这里不做解释.直接解释模型转换流程:先下载模型文件,一般是 task 格式的模型文件,这个文件可以直接解压,下面是两个下载链接:

https://storage.googleapis.com/mediapipe-models/hand_landmarker/hand_landmarker/float16/latest/hand_landmarker.task
https://storage.googleapis.com/mediapipe-models/gesture_recognizer/gesture_recognizer/float16/1/gesture_recognizer.task

下载到模型文件之后,直接解压直接使用 unzip 指令就能解压出来模型文件,因为第二个模型解压以后还是两个 task 模型,所以需要再解压一次,当前的模型文件解压出来一共 6 个模型文件,如 1 中所示文件,这里不做展示,主要是下面的模型转换,我先写出来转换的步骤,后面统一讲解代码:

```
# 默认 rknn-toolkit2 运行需要的环境是正常的
cd /home4/lhj/arm64/rockchip/airockchip/rknn-toolkit2
python3 convert_rknn_pc.py hand_detector.tflite
python3 convert_rknn_pc.py hand_landmarks_detector.tflite
python3 convert_rknn_pc.py gesture_embedder.tflite
python3 convert_rknn_pc.py canned_gesture_classifier.tflite
python3 convert_rknn_pc.py palm_detection.tflite
python3 convert_rknn_pc.py hand_landmark.tflite
```

转换的输出如下,则代表转换模型完成,会生成 6 个 rknn 模型即可



模型转换依赖于 RKNN 的 api 完成,所以必须要在 rknn-toolkit2 中进行转换,转换模型代码如下:

```
import os
import sys

_DIR = os.path.dirname(os.path.abspath(__file__))

KNOWN_NORM = {
    "palm_detection.tflite": "uint8",
```

```
    "hand_landmark.tflite": "uint8",
    "gesture_embedder.tflite": "float",
    "canned_gesture_classifier.tflite": "float",
    "hand_detector.tflite": "uint8",
    "hand_landmarks_detector.tflite": "uint8",
}

def convert(path, norm):
    from rknn.api import RKNN
    out_path = os.path.splitext(path)[0] + ".rknn"

    rk = RKNN()          # 创建对象: RKNN.__init__
    kw = dict(target_platform="rk3588")
    if norm == "uint8":
    kw.update(mean_values=[[0, 0, 0]], std_values=[[255, 255, 255]])
    if rk.config(**kw) != 0:          # 配置目标平台: rknn.config()
        print("[error] config 失败")
        sys.exit(1)
    if rk.load_tflite(path) != 0:     # 加载模型: rknn.load_tflite()
        print(f"[error] load_tflite 失败: {path}")
        sys.exit(1)
    if rk.build(do_quantization=False) != 0:       # 构建: rknn.build()
        print("[error] build 失败")
        sys.exit(1)
    if rk.export_rknn(out_path) != 0:         # 导出: rknn.export_rknn()
        print("[error] export 失败")
        sys.exit(1)
    rk.release()       # 释放资源: rknn_base.release()
    print(f"[ok] 已生成 {out_path} ({os.path.getsize(out_path) // 1024}KB)")

def main():
    a = sys.argv[1]
    p = a if os.path.exists(a) else os.path.join(_DIR, a)
    if not os.path.exists(p):
        sys.exit(1)

    norm = KNOWN_NORM.get(os.path.basename(p))
    if norm is None:
        sys.exit(1)
    convert(p, norm)


if __name__ == "__main__":
    main()
```

转换模型中,为什么要进行 kw.update()的归一处理呢?这是要从模型出发,因为这 6 个模型中有 4 个是图像模型,模型的输入参数是 uint8 的类型,两个模型是特征模型,特征模型的输入参数是 float32 类型即 0~1 的小数,所以在使用 kw.update 加入归一处理是让 npu 推理的时候,对输入的参数自动 / 255,实现归一,这样做的目的是避免 cpu 进行 / 255 的归一处理消耗时间.所以这段代码在对 uint8 类型的模型进行转换的时候,加入了 kw.update.模型转化的步骤在代码中也能清楚的看到,一共是 6 步,分别如下:

① uint8 类型的模型增加 kw.update 的归一化参数
② 设置平台:我这里是 rk3588
③ config 配置:等价于调用的 rknn.config
④ 加载模型:等价于调用的 rknn.load_tflite
⑤ 转换模型:等价于调用的 rknn.build
⑥ 输出:等价于调用的 rknn.export_rknn

3. 灵巧手控制

灵巧手的控制主要是通过发送指令来控制总线舵机.需要注意的是,在打包数据的时候,需要按照固定的帧协议进行封装,发送的数据包的格式为: FF FF FE LEN | 83 | 2A | 02 | (ID,位置高,位置低)x6 | 校验和 ,FF FF FE 是数据包的固定头,没有尾包,尾包使用检验和,中间的舵机位置的数据包是 ID 高位 低位组成,每个舵机都有三组,所以中间的位置是 18 组数据,但是发送协议的时候不一定要一次性发送 18 组,也可以只发 1 组,就代表只需要一个舵机进行运动.总线舵机在收到控制指令以后,开始运动,一般在发送控制指令的时候,舵机没有应答,在发送读取位置的指令的时候,总线舵机才会有应答数据帧,应答数据是 8 字节的数据,6 和 7 字节代表的就是舵机的位置,同样的道理,使用的是高位和低位分开字节的方式进行应答,应答数据帧如下: FF FF ID LEN 00 POS_H POS_L SUM,所以在解析的时候,对高位需要左移,比如第 6 字节和 7 字节分别为:buf[5]和 buf[6],那么应答的解析式应该是 buf[5] << 8 | buf[6],这代表把两个字节的数据整理为一个 16 位 4 字节的整形数据,就是 int 形,可以+1,则代表 1-1024 的位置信息.具体的函数如下:

```
def sync_write_frame(pos):    # 控制指令数据帧构造
    body = bytearray([0xFE, 3 + 3 * len(pos) + 1, 0x83, 0x2A, 0x02])
    for sid in sorted(pos):
        v = max(1, min(1024, int(pos[sid]))) - 1
        body += bytes([sid, (v >> 8) & 0xFF, v & 0xFF])
    return b"\xFF\xFF" + body + bytes([(~sum(body)) & 0xFF])

def parse_position(rx):    # 获取位置信息数据包构造
    if len(rx) >= 8 and rx[0:2] == b"\xFF\xFF":
        return ((rx[5] << 8) | rx[6]) + 1
    return None

def parse_position(rx): #  解析应答数据
    if len(rx) >= 8 and rx[0:2] == b"\xFF\xFF":
        return ((rx[5] << 8) | rx[6]) + 1
    return None

class CyberHand:
    def __init__(self, port):      #  串口初始化
        import serial
        self.ser = serial.Serial(port, BAUD, timeout=0.05)
        self.cur = self._read_all()                  #  从真实位置出发
        self.tgt = dict(self.cur)       # dict 是字典,键值对
        self.t_src, self.lat = None, 0.0         # t_src=当前目标来源帧的抓取时刻; lat=抓图
->实发耗时(平滑值)
        self.t_det, self.lat_gap = None, 0.0 # t_det=同帧识别结束时刻; lat_gap=识别结束
->实发(同源恒>=0)
    def _read_all(self):      #  读取 6 个舵机的初始位置
        cur = {}
        for sid in sorted(SAFE_RANGE):
            try:
                self.ser.reset_input_buffer()
                self.ser.write(read_position_cmd(sid))    #  串口发送命令
                t0 = time.time()
                pos = None
                while time.time() - t0 < 0.1:       #  等待 0.1s,100ms
                    if self.ser.in_waiting >= 8:          #  等待应答,应答 8 字节数据
                        pos = parse_position(self.ser.read(8))
                        break
                cur[sid] = pos if pos else POSE_INIT[sid] #  读位超时静默用初始位兜底
            except Exception:
                cur[sid] = POSE_INIT[sid]
        return cur
```

```
    def set_target(self, pos):
        for sid, v in pos.items():
            lo, hi = SAFE_RANGE[sid]              # 安全区间硬限制
            self.tgt[sid] = max(lo, min(hi, v))     # 限制,结合安全区间的内容进行限制
    def tick(self):
        moved = False
        for sid, c in self.cur.items():     # 遍历每个舵机的当前实际值
            d = self.tgt[sid] - c           # 目标位置 - 当前位置,等价于差值
            if abs(d) <= DEADBAND:          # 差值小于死区位置,停止即可
                continue
                step = d if abs(d) < MAX_STEP else (MAX_STEP if d > 0 else -MAX_STEP)
                self.cur[sid] = c + step
                moved = True          # 设置了 step 以后才执行运动
            if moved:
                self.ser.write(sync_write_frame(self.cur))     # 同步写广播无应答
                if self.t_src is not None: # 只测决策后第一次实发(反应时间),
                    now_ = time.time()
                    dt = now_ - self.t_src
                    self.lat = dt if self.lat <= 0 else self.lat * 0.7 + dt * 0.3
                    if self.t_det is not None:
                        dg = now_ - self.t_det
                        self.lat_gap = dg if self.lat_gap <= 0 else self.lat_gap * 0.7 + dg *
0.3         # ema 方法计算平滑参数
                    self.t_src = None
                    self.t_det = None
                if self.ser.in_waiting:
                    self.ser.reset_input_buffer()

    def move_to_init(self, timeout=2.0):
        self.set_target(POSE_INIT)
        t0 = time.time()
        while time.time() - t0 < timeout:
            self.tick()
            time.sleep(1.0 / SEND_HZ)
```

上述代码是灵巧手的运动控制的所有代码,提供给用户直接调用直接进行控制,下面对关键的几项进行解释:

① sync_write_frame() 函数,用于封装多个舵机同步运行的控制指令

② read_position_cmd() 函数用于读取单个舵机的位置的指令

封装函数,入口参数是舵机 id,所以这个函数只能调用一次读取一个舵机的角度

③ parse_position() 函数用于对发送读取位置的指令以后接收到的数据进行解包,其实就是上面描述的把 6 和 7 位的字节数据转换为一个 int 类型的数字,+1 是为了让位置信息变为 1-1024 的数值.

④ class CyberHand: 这个类,是用于在控制过程中进行舵机控制,在创建 CyberHand 的实例的时候,会通过__init__()函数初始化串口,设置串口的波特率和设备路径,我现在的 rk3588 上的路径是/dev/ttyACM0,类中的_read_al()l 函数,用于循环读取每一个舵机的真实位置信息的,set_target()函数用来设置目标位置,设置目标位置的时候,舵机并不会移动,只有发送的时候才会移动,设置只是构造数据包,类中的 tick()函数,用来让舵机运动,内部使用死区位置的方式来让舵机停止移动,只有调用 self.ser.write()这个 api,才能让舵机开始移动,类中的最后一个函数 move_to_init(),是用来恢复舵机位置的,使用设置好的默认值,来控制舵机的初始位置.

以上内容,就是灵巧手的运动控制的全部内容,总归就是一句话,控制是通过发送特定数据帧让总线舵机运动.

4. NPU 推理(palm + landmark)

代码因为太长,将通过使用附件的方式给出

毫不夸张的说,npu 推理和手势识别算法,是本项目的核心,下面我先进行 npu 推理内容的讲解.

npu 推理的对外接口是 class HandNPU 这个类下的 detect 接口,这个接口是 npu 推理的核心实现,但是在这个核心实现中也会调用几个函数来辅助 npu 推理,下面先讲解 npu 真正推理时用到的辅助函数:

① def _sigmoid(x) 函数用于输入一个 x,return 一个 0~1 之间的数值,相当于在是(-100,100)之间的数使用指数归一

② def _make_anchors() 函数是创建锚点的函数,用于在图像分析中,根据实际需求设置锚点,这里是根据模型的需要设置 2016 个锚点,创建锚点的 for fm, stride, n in ((24, 8, 2), (12, 16, 6)):参数表示创建 24*24*2+12*12*6=2016 个锚点,24 代表的像素的大小,8 代表布局,2 代表锚点数,12,16,6 同理,锚点是只有中心坐标没有宽高的,限定是 1

③ HandNPU 类在创建实例的时候,会先进行初始化,__init__()函数就是类中的初始化函数,主要是设定锚点和加载模型,创建锚点调用_make_anchors()函数,加载模型调用 self._load()函数,入口参数是模型的路径

④ self._load()函数,在 HandNPU 类中定义,实现原理是通过创建 RKNNLite()的实例,调用 load_rknn()函数,传入参数实现模型的加载,返回值为 0 代表加载成功,否则加载模型失败,然后就是初始化 npu,通过调用 init_runtime()来选择使用的 npu 核心,我这里用 NPU_CORE_0 的参数来限定 npu 核心,即 core_mask=RKNNLite.NPU_CORE_0

⑤ _split() 函数是对模型的推理结果进行输出,这个函数的调用是在模型的流水线推理中,用来输出 palm 模型的两个张量

介绍完了 npu 推理需要使用的辅助函数以后,接下来介绍主要的 npu 推理函数:

双模型的 npu 推理中,首先使用 palm 模型进行推理,这个模型的输入是 192*192 的一个图像,输出是两个张量,分别是 main 和 scores,分别表示回归张量和分数张量,回归张量的结构是[1,2016,18],2016 代表的是使用_make_anchors 创建出来的 2016 个锚点,18 代表的是每个锚点

的 18 个数据,分别是锚点中心的 x 和 y,锚点的宽和高,以及 7 个关键点的 xy 坐标,一共 18 个数据,7 个关键点分别是手腕,食指根部,中指根部,无名指根部,小拇指根部,大拇指根部和手掌.palm 模型输出两个张量以后,需要线性解码,通过解码的方式把 192*192 图像上的关键点参数回归到正式的图像中.解码完成以后,做 nms,就是获取置信度最好的手掌的信息,palm 模型本来就是用来检测手掌的,所以要使用这个置信度最高的手掌来构造下一个模型的输入图像,选出置信度最高的手掌以后,计算手腕和中指根部的连线和 y 轴的夹角,用 math.atan2()函数来计算夹角的弧度值,然后转换为角度值,使用 math.cos(theta), math.sin(theta)来计算出旋转角的正余弦值,计算的结果通过仿射矩阵来旋转和缩放,得到的结果就是 224*224 的铺满手掌的图像,并且中指正向上

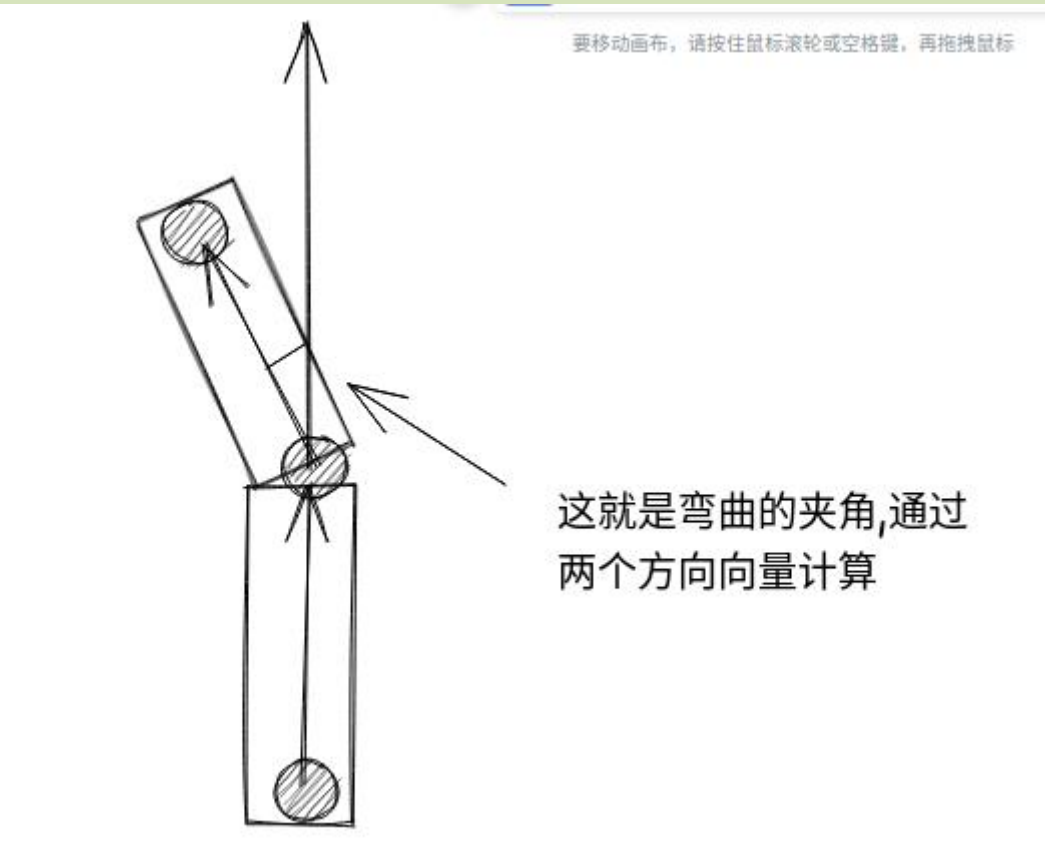
palm 模型推理结束以后,把仿射矩阵的结果给 landmark 模型进行 21 个关键点的检测,这个检测的输入是 224*224 的图像,在 palm 推理结束以后,已经在 cpu 中进行了缩放和旋转.landmark 模型的输出是 4 个张量,分别是 lm,flag,hand 和 world,分别表示局部坐标,是否是手的 flag 标志,hand 代表是左右手,world 代表是世界坐标,基本不使用,使用的是局部关键点.lm, sco2 = self._split(outs, 63)代码在模型的输出中获取局部关键点和 flag,后续进行判断如果不是手,则直接抛弃,不返回结果,如果是手,则 lm 的结果通过压缩为(21,3)的形式发给调用者进行处理,调用者就是我们的手势识别主循环代码了,这样就让识别算法和应用层的识别代码对接上了

5. 数字手势识别(0~9+ok+点赞)

手势识别中,我就不再详细的进行说明,主要介绍几个关键的函数

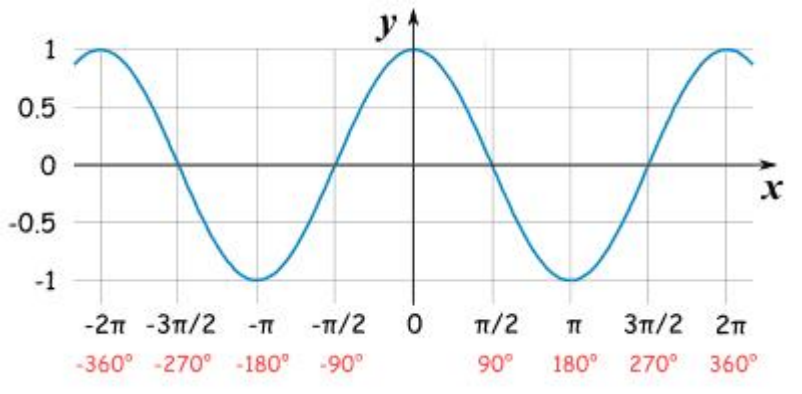
① 第一个是手势的弯曲度判断函数

```
def bend255(lm, mcp, pip, tip):
    v1 = np.subtract(lm[pip], lm[mcp]); v2 = np.subtract(lm[tip], lm[pip])
    c = np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2) + 1e-6)
    return int(np.clip(np.degrees(np.arccos(np.clip(c, -1, 1))) / 120.0, 0, 1) * 255)
```



在手势的弯曲度判别函数中,主要是对 python 的 numpy 库的使用,和高中数学的向量功底.首先计算的是 v1 和 v2,这是一根手指上确定下来的两个方向向量,然后 dot 函数用来计算两个向量的点积,继续使用 norm 计算向量的模,最后使用两个向量的点积和模的公式,计算出两向量之间的夹角的余弦值,这里为什么要计算余弦值,这就是高中数学的 0-180 度之间的余弦的单调递减的性质,用正弦值就不准确,得到

余弦值以后,使用 arccos 这个反余弦函数计算出弧度值,然后弧度值转换为角度值,最后把角度值用 120 度进行归一,这是因为一般手指不可能弯曲大于 120 度,归一后的结果又乘 255,目的是让 0-255 代表手指的弯曲度,短短 3 行代码,全是数学知识



② classify () 数字分类函数

```
def classify(lm):
    b = {k: bend255(lm, m, p, t) for k, (m, p, t) in FINGERS.items()}
    ext = {k: v < 90 for k, v in b.items()}
    bnt = {k: v > 128 for k, v in b.items()}
    palm = np.hypot(lm[9][0] - lm[0][0], lm[9][1] - lm[0][1]) + 1e-6
    def d(i, j):
        return float(np.hypot(lm[i][0] - lm[j][0], lm[i][1] - lm[j][1]) / palm)
    fold, spread = d(4, 17), d(4, 5)          # 拇指尖-小指根 / 拇指尖-食指根
    pin_i, pin_m = d(4, 8), d(8, 12)          # 拇食指尖 / 食中指尖
    tb = bend255(lm, 2, 3, 4)
    h_ir = d(8, 16)
    m_r = d(12, 16)
    pin3 = max(pin_i, pin_m, d(4, 12))        # 拇/食/中三指尖(4,8,12)最大间距
```

上面贴出来的代码,是数字分类的核心数据,通过 palm 参数来保存手掌的长度,就是手腕到中指根部的距离,函数是 hypot 函数,入口参数是 dx,dy,结果就是两点间的距离.定义的 d 这个函数,是计算的两点间的距离相对于掌长的距离,计算的是相对值.下面是每个数字的特征,用于判断手势所作的数字

③ 数字 7: 拇/食/中三指尖最大间距小于 0.4 ,就是接近三指聚在一起,中指尖到无名指尖距离大于 0.45,说明这两个指尖是分开的

④ 手势 ok:中、无名、小指伸直处于伸直状态,食指弯曲,拇指和食指尖距离小,相当于捏合

⑤ 数字 9: 四个手指全弯曲,食指尖到中指尖的距离大

⑥ 点赞: 四指全弯曲,大拇指翘起

⑦ 数字 10: 四指全弯,不是 9 也不是点赞

⑧ 数字 4 和 5: 判断大拇指弯曲与否

⑨ 数字 3: 食指中指无名指伸直,小拇指和大拇指弯曲

⑩ 数字 2: 两个伸直,三个弯曲

⑪ 数字 1: 只有食指伸直,其余弯曲

控制中,因为代码中已经固定了这些动作的位置信息,所以识别到之后,直接发送指令,进行模拟即可

6. 手势跟随

手势跟随中,相比于数字识别,多了一个 thumb_255 函数,这个函数用来区分大拇指的张开和弯曲,代码如下:

```
def thumb_255(lm):      # 判断拇指的弯曲和张开
    palm = np.hypot(lm[9][0] - lm[0][0], lm[9][1] - lm[0][1]) + 1e-6    # 腕-中指根 = 手掌长
    fold = np.hypot(lm[4][0] - lm[17][0], lm[4][1] - lm[17][1]) / palm        # 拇指尖-小指根
    spread = np.hypot(lm[4][0] - lm[5][0], lm[4][1] - lm[5][1]) / palm        # 拇指尖-食指根
    THUMB_RAW["fold"], THUMB_RAW["spread"] = round(fold, 2), round(spread, 2)
    bend = int(np.clip((1.05 - fold) / 0.65, 0, 1) * 255)      # fold 1.05->直 0, 0.40->全弯 255
    rot = int(np.clip((spread - 0.25) / 0.30, 0, 1) * 255) # spread 0.25->合拢
    return bend, rot        # 返回  (bend255, rot255): bend 0 直~255 弯, rot 0 合拢~255 张开
```

这个函数的关键是通过距离信息来判断大拇指的弯曲和张开,标准依然是手掌的长,就是手腕到中指根部的距离作为基准,一共测两个距离 fold 是大拇指尖到小拇指根的距离,spread 是大拇指尖到食指根的距离,通过这两个距离的大小,判断拇指的弯曲程度,fold 越小,说明大拇指越弯,spread 越大,说明大拇指越直.

除此以外,还有一个关键的函数,就是用来把 21 个点的坐标转换为舵机的位置信息的,如下

```
def retarget_pos(lm):
    bends = {k: bend255(lm, m, p, t) for k, (m, p, t) in FINGERS.items()} #计算 4 指的弯曲度
    bends["thumb_bend"], bends["thumb_rot"] = thumb_255(lm)   # 计算拇指的弯曲度
    pos = {}
    for sid, (key, straight, bent) in MAP.items():     # 遍历映射表
        t = bends[key] / 255.0   # 手指的弯曲程度归一化: bends[key] / 255.0
        pos[sid] = round(straight + (bent - straight) * t)       # 线性插值: t = 0,则伸直用
map 表中的伸直数,t=1 则弯曲吗,用 round 转为整数
    return pos
```

其实这段代码的核心是调用函数获取弯曲程度,然后归一化,归一结束使用线性插值的方式设置目标位置,原理如下:

先算出每根手指的弯曲程度——四指用 bend255 比较两段指骨的夹角，拇指用 thumb_255 比较拇指尖到小指根、食指根的距离比，统一得到 0 - 255 的弯曲度；然后查 MAP 映射表，每个舵机都预设了伸直和弯曲两个端点，把弯曲度归一化成 0~1 的比例 t，用 straight + (bent - straight) * t 做线性插值——手伸直（t=0）舵机就到伸直位，手全弯（t=1）就到弯曲位，弯一半就走一半，最终得到 6 路舵机的目标位置字典 {1: 位置, ..., 6: 位置}，交给 set_target 限速逼近执行

7. NPU 推理(palm+landmark+emb+classify)

这次介绍的是 4 个模型的流水线作业,使用 4 个模型的配合推理,实现 7 中固定手势的分类.首先介绍一下 4 个模型的推理流程和模型的输入输出形式,先从模型的推理流程开始,我会把关键步骤和推理流程都写在下面,推理只是调用 rknn 模型在 npu 上做运算,运算的结果和衔接,还需要在 cpu 上手动实现进行传递:

① 打开摄像头取图,默认取图是 640*480 的 4:3 图像

② 图像转换为 192*192 的图像,因为 palm 模型的输入参数是 192*192

③ 转换后的图像,在图上创建锚点,一共创建 2016 个

④ 第 3 步中的结果作为 palm 的输入参数,让 npu 进行推理,输出是 2 个张量,回归张量和分数张量,这部分内容在 NPU 推理(palm + landmark) 中已经介绍

过,不做重复的介绍

⑤ 输出的结果不能直接给 landmark 模型,所以需要进行线性解码,线性解码的意思就是把 192*192 图像检测到的手掌坐标,映射回原图,然后根据分数张量,倒序排列,计算 iou,iou 就是两个框的交并比,用来选出重复的手掌框,在循环的最后,只剩下一个分数最大的手掌框,这就是我们需要的

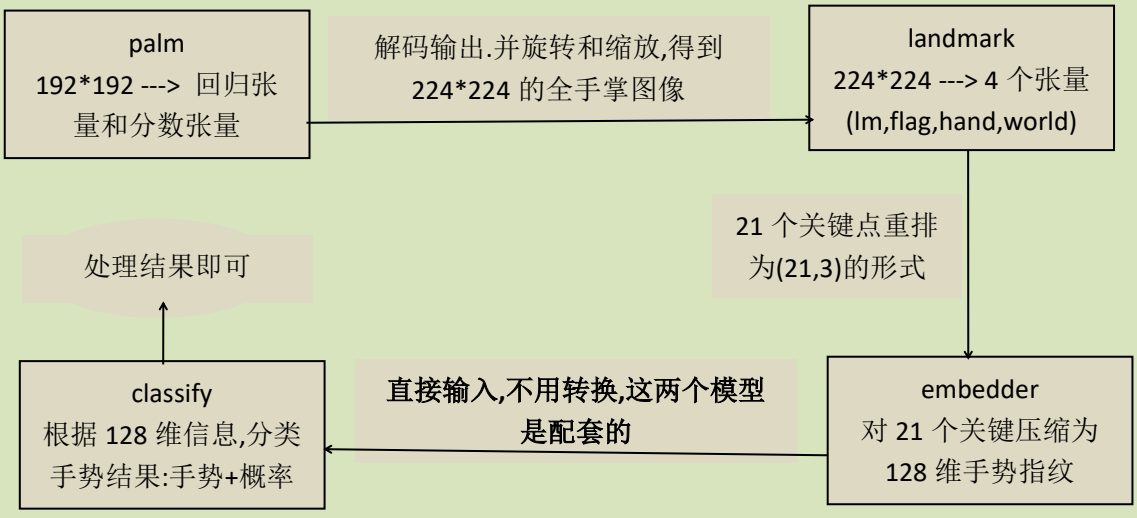
⑥ 对最大的手掌进行旋转和放大,目的是把倾斜的手掌转正,让手腕和中指根的连线在 y 轴的这条轴线上,放大是让构造出的 224*224 的图像中出现手掌+手指,而不是只有手掌,这部分通过计算缩放因子,旋转角,然后利用仿射矩阵实现

⑦ 在 6 中构造出来的 224*224 的图像,就是 landmark 模型的输入参数,模型的输出是 4 个张量,分别是 lm,flag,hand,world,在这 4 个张量中,只需要使用两个,分别是 lm 和 flag,lm 存放的 2016 个锚点所的对应的 21 个关键点的坐标(x,y,z),flag 对应的是是否是手的标志

⑧ 对 landmark 模型的输出,需要把 21 个关键点的信息传递给 gesture_embedder.rknn 模型,输出的 21 个关键点的信息通过 np.stack 排列为 21 个关键点的(x,y),取了 z 坐标,并且除原图大小来对这些坐标归一化,归一化之后的结果,就可以通过 embedder 模型后输出的是 21 个关键信息的 xyz 压缩的 128 维的手势指纹

⑨ embedder 模型输出的 128 维的手势指纹,已经是 float 类型,不用转换,可以直接给 classify 模型进行分类.手势分类的模型的输出就是 8 个手势的概率和 logit,logit 代表是什么手势,还有一个就是这个手势对应的概率,应用中选择概率最大的使用即可

⑩ 注意事项: 因为识别的结果中,把 V 手势和点赞手势区分的不是很好,所以在算法中采用几何增强的方式处理,这部分内容在具体的分类章节讲解



8. 固定手势识别

固定手势识别,主要是对 classify 模型的输出结果进行处理,因为 4 个模型的流水线推理,已经可以正常输出手势的类型和概率,所以在对结果进行处理的时候,难度并不大,相对而言,难度集中在 4 个模型的 npu 推理部分.先介绍几个相关的函数:

① def geo_victory(lm) 函数,用于判断固定手势中的"V"的手势,函数如下:

```
def geo_victory(lm):       # 判断耶的手势
    try:
        b_i = bend255(lm, 5, 6, 8)       # 食指
        b_m = bend255(lm, 9, 10, 12)    # 中指
        b_r = bend255(lm, 13, 14, 16) # 无名指
        b_p = bend255(lm, 17, 18, 20) # 小拇指
    except Exception:
        return False
    return b_i < 165 and b_m < 165 and b_r > 185 and b_p > 185        # V 手势,输出 true
```

bend255() 函数的实现,已经在前面的内容中详细介绍,这里不再介绍,这个函数就是判断手指的弯曲度的,输出是(0,255),数值越大越弯曲.现在 V 手势的判定条件是食指和中指伸开,无名指和小拇指弯曲,则是 V 的手势

② 主函数中的几何增强

```
# 几何补强的方式
if geo_streak >= 2:
    pb[6] = max(pb[6], GEO_V_BOOST)
    pb[5] = min(pb[5], 0.20)
ema = (1.0 - EMA_A) * ema + EMA_A * pb
j = int(np.argmax(ema[1:])) + 1
g = GESTURES[j]
if last_g != g and ema[j] >= (ADOPT_V if j == 6 else ADOPT):
    last_g = g
    if hand is not None:
        hand.t_src = t_grab
        hand.t_det = t_det_end
                hand.set_target(POSES[g])
                next_send = 0.0
```

何为几何增强?就是代码中设置某一个手势的概率,来提高这个手势被识别

的概率,那么为什么这么处理,完全是因为模型识别出来的 v 手势和点赞的手势识别率太低了,所以在代码中如果连续两个都是 v 手势,就主动把 v 手势的概率增大到 0.75,同时把点赞手势的概率降低到 0.2,这样的处理就在一定程度上提高了手势的识别率.4 模型固定手势识别的主循环识别代码如下:

```
try:
    while True:
        if STATE == "running" and prev_state != "running":     # 清防抖状态,
            ema[:] = 0
            geo_streak = 0
            last_g = None
        prev_state = STATE
        if STATE == "paused":     # 暂停: 不抓图不识别, 最后一帧定格显示
            if last_img is None:
                continue
            img = last_img
        else:
            ok, img = cap.read()
            if not ok: continue
            t_grab = time.time()     # t_grab 抓到图片的时间点
            n += 1
            img = cv2.rotate(img, cv2.ROTATE_180)
            if STATE == "running":                    # idle 只预览画面, 不跑推理省 NPU
                res = landmarker.detect(img)
                t_det_end = time.time()
                dec = t_det_end - t_grab     # dec 代表 推理时间
                dec_lat = dec if dec_lat <= 0 else dec_lat * 0.7 + dec * 0.3
                lm = res["pts"] if res else None     # 推理成功
                if lm:
                    h, w = img.shape[:2]
                    pts = [(int(p[0]*w), int(p[1]*h)) for p in lm]
                    for a, b in CONNS: cv2.line(img, pts[a], pts[b], (0,255,0), 2)
                    for p in pts: cv2.circle(img, p, 3, (0,0,255), -1)        # 红点标记
                    if res["probs"] is not None:        # 包含 8 个手势概率结果
                        pb = np.asarray(res["probs"], np.float32).copy()
                        geo_streak = geo_streak + 1 if geo_victory(lm) else 0
                        # 几何补强的方式
                        if geo_streak >= 2:
                            pb[6] = max(pb[6], GEO_V_BOOST)
                            pb[5] = min(pb[5], 0.20)
                        ema = (1.0 - EMA_A) * ema + EMA_A * pb
                        j = int(np.argmax(ema[1:])) + 1
                        g = GESTURES[j]
```

```
                                g = GESTURES[j]       # g 代表手势
                                if last_g != g and ema[j] >= (ADOPT_V if j == 6 else ADOPT):
                                    last_g = g
                                    if hand is not None:
                                        hand.t_src = t_grab       # 记录采集到图片时刻
                                        hand.t_det = t_det_end # 采集推理结束的时间点
                                        hand.set_target(POSES[g])     # 只更新目标
                                        next_send = 0.0
                                elif last_g is not None and ema[GESTURES.index(last_g)] <
RELEASE:       # 撤回手势: 手势的概率 < 0.18,则撤回这个手势
                                    last_g = None                    # 撤标签; 姿势保持不动
                            else:   # 没有手的时候: 清防抖, 停发新指令, 姿势保持
                                if last_g is not None:
                                    last_g = None
                                ema[:] = 0
                                geo_streak = 0
                    if hand is not None and STATE == "running" and time.time() >= next_send:
                        try:
                            hand.tick()   # 限速逼近目标
                        except Exception as e:
                            print("[hand] 串口异常, 停止控制:", e, flush=True)
                            break
                        next_send = time.time() + 1.0 / SEND_HZ
                    now = time.time()
                    if now - t_last >= 1.0:
                        fps = fps * 0.5 + (n / (now - t_last)) * 0.5
                        n, t_last = 0, now
                    if now - t_report >= 5.0:
                        dec = f"{dec_lat*1000:.0f}ms" if dec_lat > 0 else "--ms"
                        if hand is None:                                # dry-run: 没有串口实发
                            print(f"[status] {fps:.1f} fps | 抓图->识别 {dec}", flush=True)
                        else:
                            gap = f"{hand.lat_gap*1000:.0f}ms" if hand.lat_gap > 0 else "--ms"
                            print(f"[status] {fps:.1f} fps | 抓图->识别 {dec} | 识别->发送 {gap}",
flush=True)
                        t_report = now
                    bw = (img.shape[0] * SW - img.shape[1] * SH) // (2 * SH)
                    if bw > 0:
                        img = cv2.copyMakeBorder(img, 0, 0, bw, bw, cv2.BORDER_CONSTANT,
value=(0, 0, 0))
                    draw_buttons(img)          # 屏上按钮条
                    show_msg = "PAUSED" if STATE == "paused" else ("PRESS START TO PLAY" if STATE
== "idle"
```